Table of Contents

Introduction **2**

Three ways of Conditional rendering **3**

Splitting the component from the page **4**

Conditional rendering of ItemGrid or ItemTable *6*

Wait on the lagging backend *11*

Toggling descriptions and images in ItemTable **12**

Filtering content **14**

Carousel in HomePage **16**

Demo: Conditional Rendering

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Gemini.

Introduction

Conditional rendering in React enables dynamic control over what gets displayed to the user based on state, props, or other criteria. This demo explores three primary methods of achieving conditional rendering:

- **If Statement**: Traditional if statements determine which set of JSX elements to render based on conditions.
- **Ternary Operator**: This concise approach uses the format condition ? true : false to render elements based on a condition. For example, button text might change depending on a loading state.
- **Logical AND (&&)**: The logical AND operator renders content only if a condition is true, such as displaying an error message when an error value is present.

Splitting the Component

To enhance clarity, the demo splits ItemListPage into the ItemTable component, responsible only for the ItemLable, and the ItemListPage component, responsible for the page.

ItemGrid Component

A new component, ItemGrid, is added alongside ItemTable. ItemListPage.jsx now conditionally renders either ItemTable or ItemGrid based on a state variable. This approach involves:

Toggling Columns in ItemTable

Additional functionality is introduced to ItemTable.jsx to toggle the visibility of item descriptions and images.

Filtering Content

A search bar is integrated to filter items based on the search query. Changes to ItemListPage.jsx include:

Carousel in HomePage

Finally, a carousel component is added to HomePage.jsx using react-bootstrap.

Three ways of Conditional rendering

Conditional rendering in React refers to the practice of rendering different elements or components based on certain conditions. It allows you to dynamically control what gets displayed to the user based on the state, props, or other criteria. We have seen this a bit in our previous demo of Http Get, and will see more in this demo.

There are three ways to achieve it:

if Statement: You can use traditional if statements to conditionally render elements. For example, in a function, you might return one set of JSX if a condition is true, and another set if it is false. Example:

```
function MyComponent({ isLoggedIn }) {
  if (isLoggedIn) {
    return <h1>Welcome back!</h1>;
  } else {
    return <h1>Please sign up.</h1>;
  }
}
```

```
18    const { data: items = [], isLoading, error, refetch } = useQuery({
19      queryKey: ['items'],
20      queryFn: fetchItems,
21      if (error) { // for local development debugging, remove for production
22      console.error(`There was a problem with the fetch operation: ${error.message}`);
23    }
```

This is conditional console logging, not conditional rendering, but the principle is similar.

Ternary Operator: A more concise way is using the ternary operator
(condition ? true : false).
In ItemListPage.jsx: changing the button text depending on the Boolean value of isLoading

```
29        <Button onClick={() => refetch()} className="btn btn-primary mb-3" disabled={isLoading}>
30          {isLoading ? 'Loading...' : 'Refresh Items'}
31        </Button>
```

Logical AND (&&): This operator is used to render something only if a condition is true. It is useful for simple conditions.
In ItemListPage.jsx: display the error message if the error value is not false.

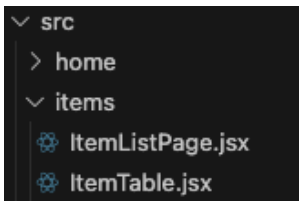
```
32        {error && <p style={{ color: 'red' }}>Failed to fetch items.</p>}
```

Splitting the component from the page

Now we are going to add a new component, ItemGrid. The ItemListPage.jsx should display the items in either Table or Grid, depending on a state.

Before adding ItemGrid, we first need to split the current code of ItemListPage.jsx into: (1) a page component that is responsible for the rendering and the logic for the ItemListPage, and (2) a Table component that is only responsible for the ItemTable.

We create a file ItemTable.jsx under myshop/src/items



Change the ItemTable.jsx to:

ItemTable.jsx U

6-React-Intro > Demo-react-3-httpget > myshop > src > items > ItemTable.jsx > ItemTable

```jsx
import { Table } from 'react-bootstrap';

const ItemTable = ({ items, apiUrl }) => {

  return (
    <Table striped bordered hover>
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Price</th>
          <th>Descriptions</th>
          <th>Images</th>
        </tr>
      </thead>
      <tbody>
        {items.map(item => (
          <tr key={item.itemId}>
            <td>{item.itemId}</td>
            <td>{item.name}</td>
            <td>{item.price} NOK</td>
            <td>{item.description}</td>
            <td><img src={`${apiUrl}${item.imageUrl}`} alt={item.name} width="120" /></td>
          </tr>
        ))}
      </tbody>
    </Table>
  );
};

export default ItemTable;
```

We can see that the ItemTable now becomes a simple component, which takes two input arguments items and apiUrl.

Change the ItemListPage.jsx to:

ItemListPage.jsx U

6-React-Intro > Demo-react-4-1-conditionalrendering > myshop > src > items > ItemListPage.jsx > API_URL

```jsx
import { useQuery } from '@tanstack/react-query';
import { Button } from 'react-bootstrap';
import ItemTable from './ItemTable';

const API_URL = 'http://localhost:5234'

const fetchItems = async () => {
  const response = await fetch(`${API_URL}/api/itemapi/itemlist`);
  if (!response.ok) {
    console.error(`Fetch failed with status: ${response.status}`);
    throw new Error('Network response was not ok');
  }
  const data = await response.json();
  console.log('Fetched items successfully:', data); // remove for production
  return data;
};

const ItemListPage = () => {
  const { data: items = [], isLoading, error, refetch } = useQuery({
    queryKey: ['items'],
    queryFn: fetchItems,
  });

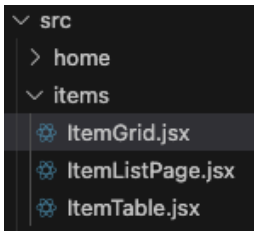
  return (
    <div className="text-start">
      <h1>Items</h1>
      <Button onClick={() => refetch()} variant="primary" className="mb-3 me-2" disabled={isLoading}>
        {isLoading ? 'Loading...' : 'Refresh Items'}
      </Button>
      {error && <p style={{ color: 'red' }}>Failed to fetch items.</p>}
      <ItemTable items={items} apiUrl={API_URL} />
    </div>
  );
};

export default ItemListPage;
```

Import ItemTable: imports the ItemTable component from the ./ItemTable.jsx file. The jsx extension is omitted because modern JavaScript tooling (e.g., Webpack, Babel) and module bundlers are configured to automatically resolve file extensions.
Line 31 uses the ItemTable component, passing two arguments into it.
Now we see that the long file ItemListPage.jsx becomes much clear. This separation helps maintain a clean structure and promotes better reusability and readability.
The effect of this code is exact the same as previous code and there will be no visible difference.

Conditional rendering of ItemGrid or ItemTable

Now we add a new file ItemGrid.jsx under myshop/src/items



Change it to:

ItemGrid.jsx U

6-React-Intro > Demo-react-4-1-conditionalrendering > myshop > src > items > ItemGrid.jsx > ...

```jsx
import { Card, Col, Row } from 'react-bootstrap';

const ItemGrid = ({ items, apiUrl }) => {

  return (
    <div>
      <Row xs={1} sm={2} md={3} lg={4} className="g-4">
        {items.map(item => (
          <Col key={item.itemId}>
            <Card className="h-100">
              <Card.Img variant="top" src={`${apiUrl}${item.imageUrl}`} alt={item.name} />
              <Card.Body>
                <Card.Title>{item.name}</Card.Title>
                <Card.Text>
                  {item.description}
                </Card.Text>
                <Card.Text>
                  Price: {item.price} NOK
                </Card.Text>
              </Card.Body>
            </Card>
          </Col>
        ))}
      </Row>
    </div>
  );
};

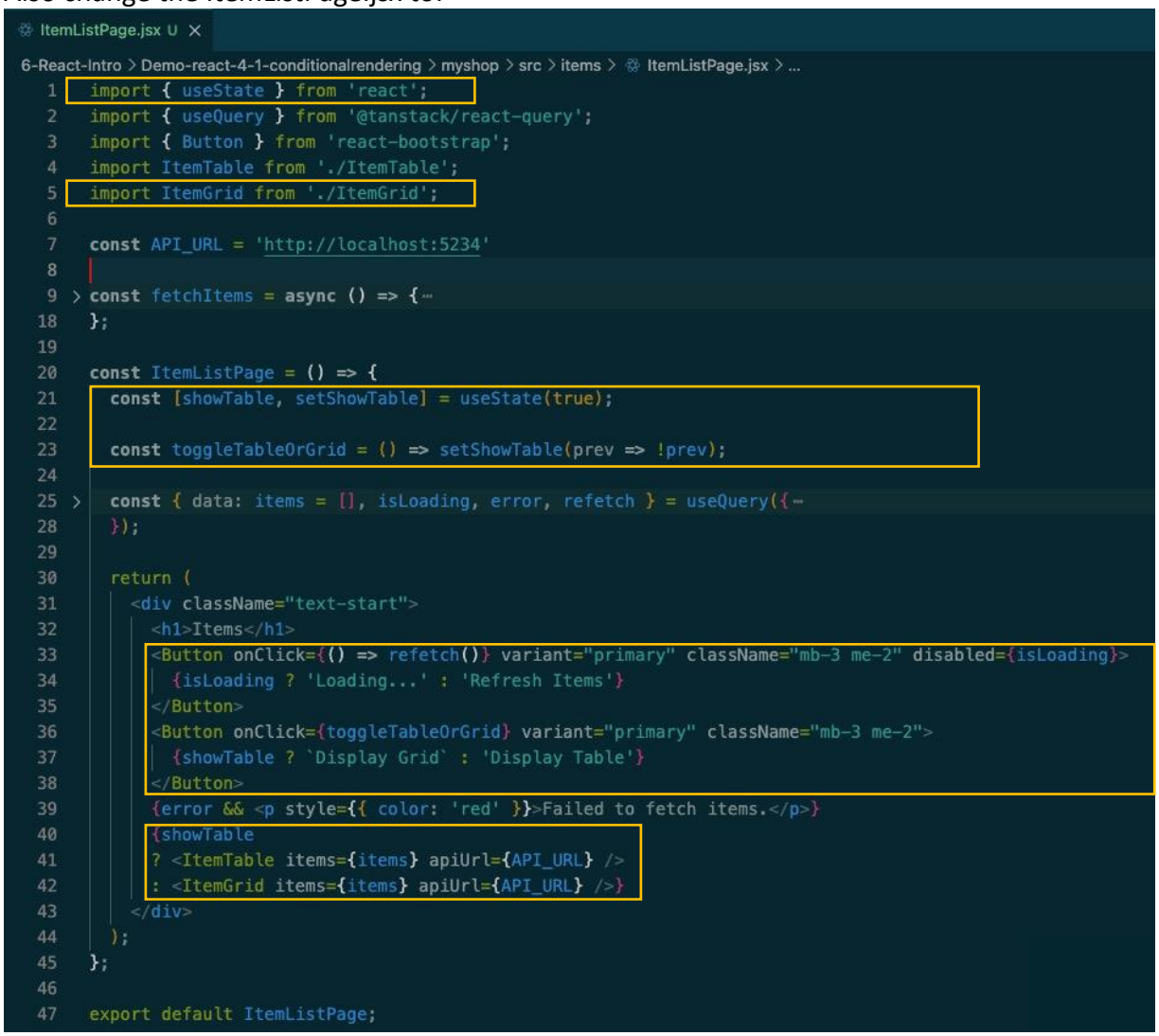
export default ItemGrid;
```

Imports: Card, Col, Row are components from the react-bootstrap library used for layout and styling.
const ItemGrid defines a functional component that takes items and apiUrl as input arguments (common referred to as properties, or props, in React).
<Row>: A Bootstrap component that creates a row in the grid. The responsive props (xs, sm, md, lg) define how many columns the row should have at different screen sizes.
<Col>: A Bootstrap component that represents a column within a row. Each item is placed in its own column.
<Card>: A Bootstrap component used to display content in a card format. Each item is represented as a card with an image, title, description, and price.
export default ItemGrid: Exports the ItemGrid component as the default export from this module, allowing it to be imported and used in other parts of the application.

<Card.Body>: Contains the main content of the card, including:

- **<Card.Title>**: Displays the item's name.
- **<Card.Text>**: Displays additional text, including description and price.

Also change the ItemListPage.jsx to:



```jsx
import { useState } from 'react';
import { useQuery } from '@tanstack/react-query';
import { Button } from 'react-bootstrap';
import ItemTable from './ItemTable';
import ItemGrid from './ItemGrid';

const API_URL = 'http://localhost:5234'

const fetchItems = async () => {…
};

const ItemListPage = () => {
  const [showTable, setShowTable] = useState(true);

  const toggleTableOrGrid = () => setShowTable(prev => !prev);

  const { data: items = [], isLoading, error, refetch } = useQuery({…
  });

  return (
    <div className="text-start">
      <h1>Items</h1>
      <Button onClick={() => refetch()} variant="primary" className="mb-3 me-2" disabled={isLoading}>
        {isLoading ? 'Loading...' : 'Refresh Items'}
      </Button>
      <Button onClick={toggleTableOrGrid} variant="primary" className="mb-3 me-2">
        {showTable ? `Display Grid` : 'Display Table'}
      </Button>
      {error && <p style={{ color: 'red' }}>Failed to fetch items.</p>}
      {showTable
      ? <ItemTable items={items} apiUrl={API_URL} />
      : <ItemGrid items={items} apiUrl={API_URL} />}
    </div>
  );
};

export default ItemListPage;
```

Line5 imports the **ItemGrid** component.
Line21 adds a state "**showTable**" for showing the **ItemTable** (when true) or the **ItemGrid** (when false).

useState Hook
The useState hook allows you to add state to a functional component. It returns an array with two elements:

- The current state value.
- A function to update that state.

Line23 adds a function that toggles the true/false value of **showTable**.
Why developers choose const arrow functions for helper logic:

1. **Consistency with callbacks:** Arrow functions use the `() => {}` syntax, matching how inline event handlers and array methods (`.map()`, `.filter()`) are written in React.

2. **Preventing accidental reassignment:** Using const guarantees that the variable name cannot be reassigned elsewhere in the block.
3. **Implicit Lexical Scope (this):** Arrow functions do not bind their own this keyword, making them ideal for inner event handlers and inline callbacks.

Note: For top-level React components, both `function ComponentName() {}` *and* `const ComponentName = () => {}` *are widely used and functionally equivalent in modern JavaScript and TypeScript.*

When updating React state based on its current value, use the function form: `setShowTable(prev => !prev)`. React automatically passes the current state as the first parameter (prevShowTable - you can name it anything). This ensures you always get the latest state value, preventing bugs from rapid state updates. Use this pattern whenever your new state depends on the previous state.

The **fetchItems** (line9) and **useQuery** (line25) are unchanged.
Line39 – Line41 adds an extra Button, clicking which will trigger the **toggleTableOrGrid** function. The button text is conditional rendered depending on the showTable variable.
Line43 – Line45 conditional renders the **ItemTable**, when **showTable** is true, or the ItemGrid, when **showTable** is false.
Now we run the project.

- Go to the terminal, `cd myshop`
- `npm install`
- `npm run dev`

The webapp is running as we expected:

My Shop

http://localhost:5173/items

Items

Refresh Items | Display Table



Pizza

Delicious Italian dish with a thin crust topped with tomato sauce, cheese, and various toppings.

Price: 150 NOK



Fried Chicken Leg

Crispy and succulent chicken leg that is deep-fried to perfection, often served as a popular fast food item.

Price: 20 NOK



French Fries

Crispy, golden-brown potato slices seasoned with salt and often served as a popular side dish or snack.

Price: 50 NOK



Grilled Ribs

Tender and flavorful ribs grilled to perfection, usually served with barbecue sauce.

Price: 250 NOK



Tacos

Tortillas filled with various ingredients such as seasoned meat, vegetables, and salsa, folded into a delicious handheld meal.

Price: 150 NOK



Fish and Chips

Classic British dish featuring battered and deep-fried fish served with thick-cut fried potatoes.

Price: 180 NOK



Cider

Refreshing alcoholic beverage made from fermented apple juice, available in various flavors.

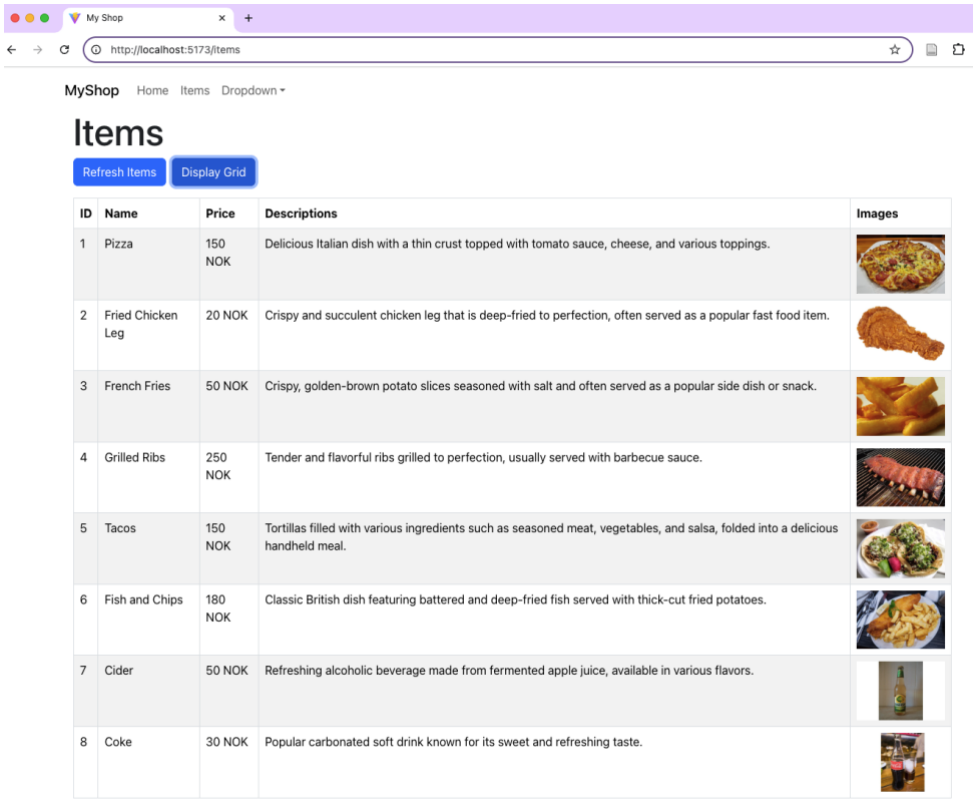
Price: 50 NOK



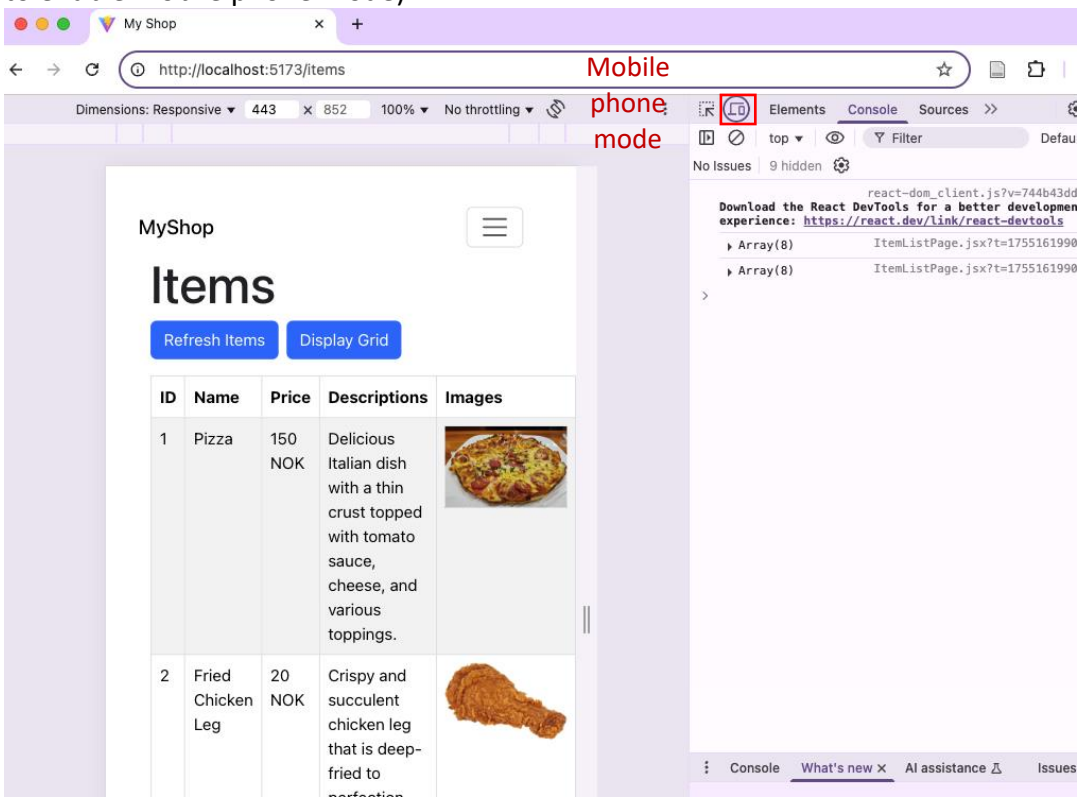
Coke

Popular carbonated soft drink known for its sweet and refreshing taste.

Price: 30 NOK



However, it may look not so nice for the table if we are using a small screen (click the button to enable mobile phone mode):



Also, we would like to add a search mechanism if the item list is too long.
We do these in the next section.

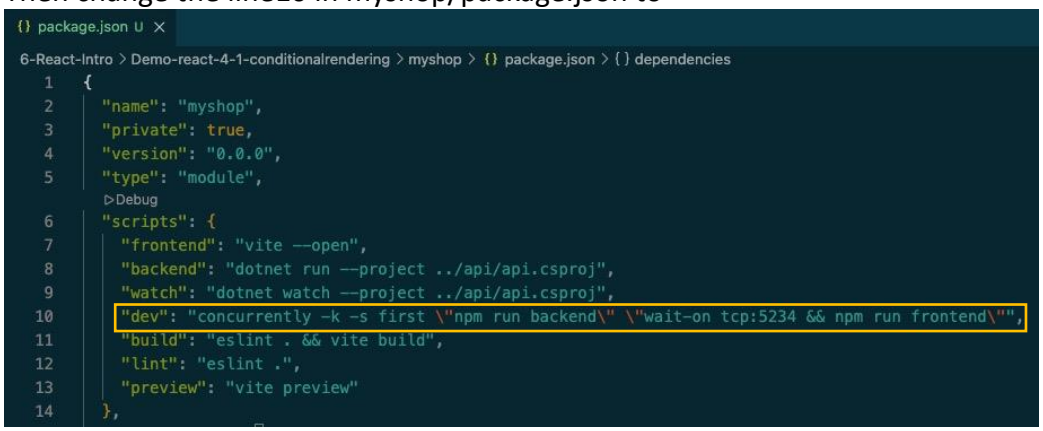
Wait on the lagging backend

You may notice that sometimes your fetch items will fail because the frontend is running before your backend is ready. A simple fix is to let the frontend wait until the backend is ready. This is typically used in a development environment.

Go to myshop

```
cd myshop
npm install --save-dev wait-on
```

Then change the line10 in myshop/package.json to



This line of code make the frontend to start after waiting for the port to be open, regardless of HTTP responses.

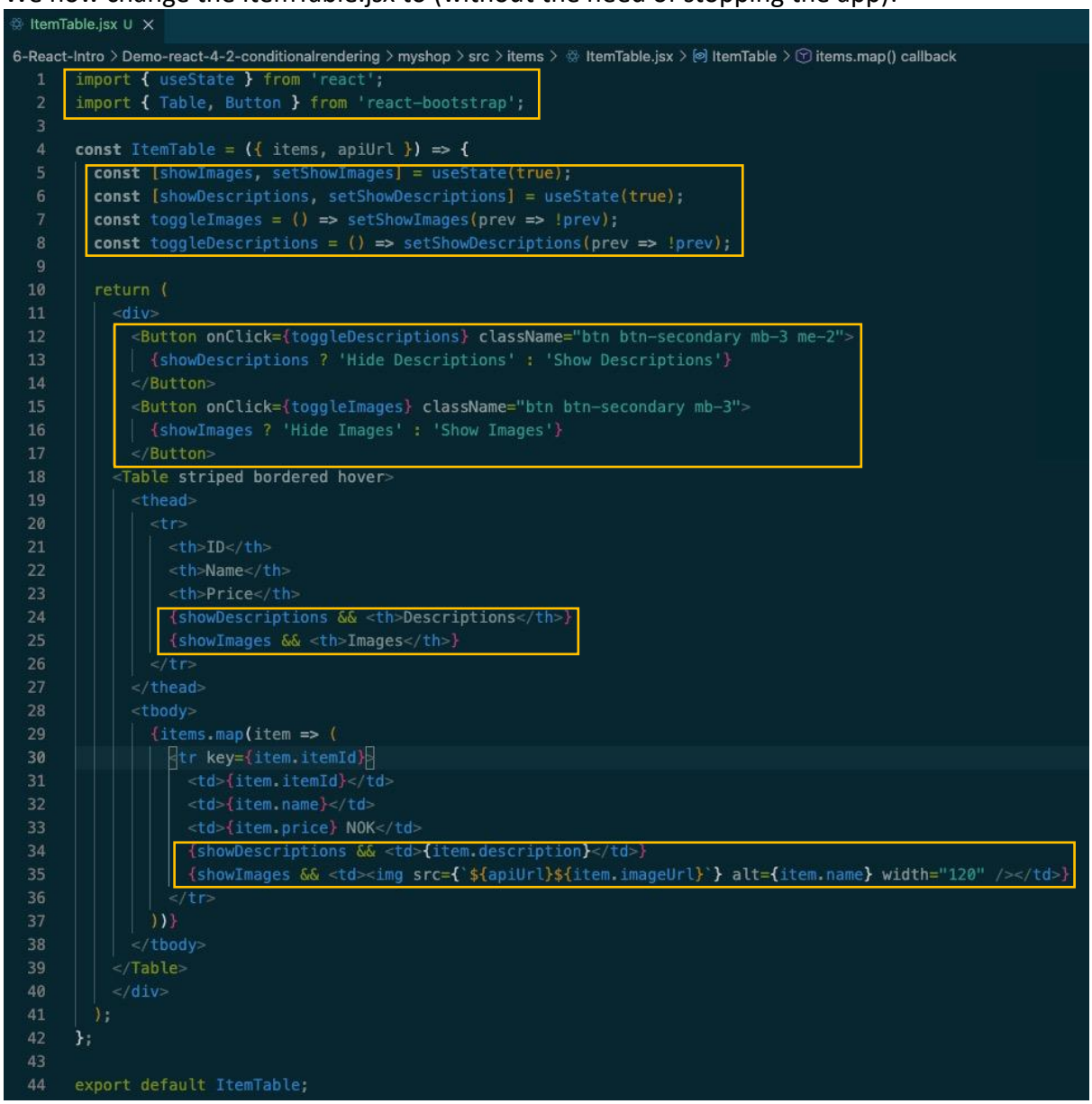
Then run

```
npm run dev
```

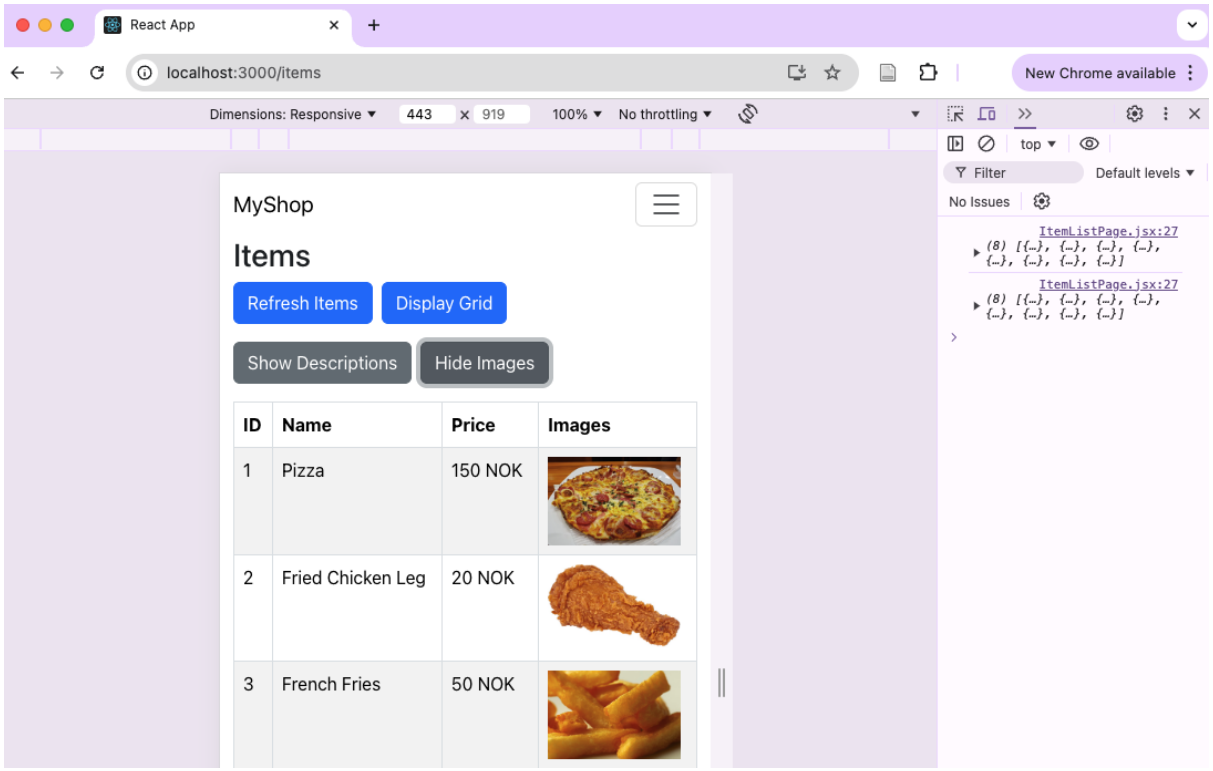
You will see that the frontend always starts after the backend is ready.

Toggling descriptions and images in ItemTable

We now change the ItemTable.jsx to (without the need of stopping the app):

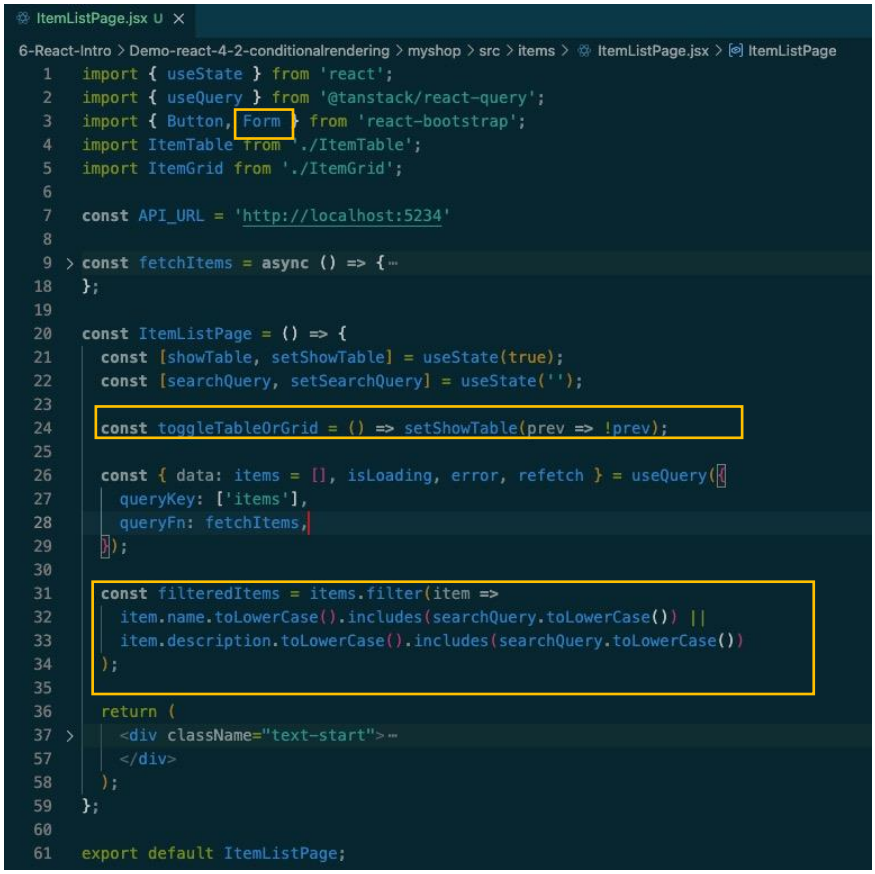


Imports: import the hook **useState** and component Button
Line5-Line6: define two new states for toggling the images and descriptions.
Line7-Line8: define two functions for the togging true/false value of **showImages** and **showDescriptions**.
Line12-Line15: adding two buttons for toggling.
Line24-Line25: toggling the table headers.
Line34-Line35: toggling the table data.
Line11 and Line40: wrapping up the buttons and tables into a , required by the return command.
We can see the effect immediately:


React App
localhost:3000/items
New Chrome available
Dimensions: Responsive
443
919
100%
No throttling
top
Filter
Default levels
No Issues
ItemListPage.jsx:27
(8) [{…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}]
ItemListPage.jsx:27
(8) [{…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}]
MyShop
Items
Refresh Items
Display Grid
Show Descriptions
Hide Images
ID
Name
Price
Images
1
Pizza
150 NOK
2
Fried Chicken Leg
20 NOK
3
French Fries
50 NOK

Filtering content

Now we add a search bar before the ItemTable or ItemGrid.
Change the ItemListPage.jsx to:
Line1-Line45



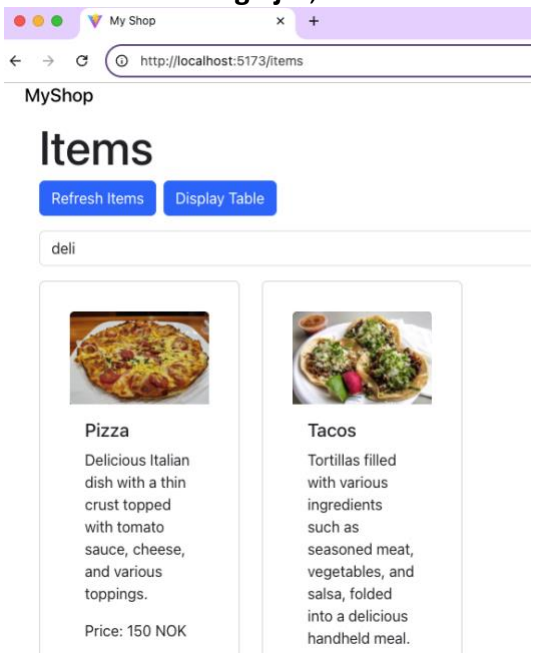
Line3: imports **Form**, we need for inputting search query.
Line22: defines a new state, **searchQuery**.
Line9, Line26, no change.
Line31-Line34: defines a new const to store the result of the filtering operation. It calls the filter method on the items array, which creates a new array with all elements that pass a test implemented by the provided function.
item =>: Uses an arrow function as the test function for filtering. Each item in the items array is passed to this function.
The **toLowerCase()** makes the search case insensitive.
The || is logical OR operator, it means either the item.name or the **item.description** needs to contain the string in the **searchQuery**.

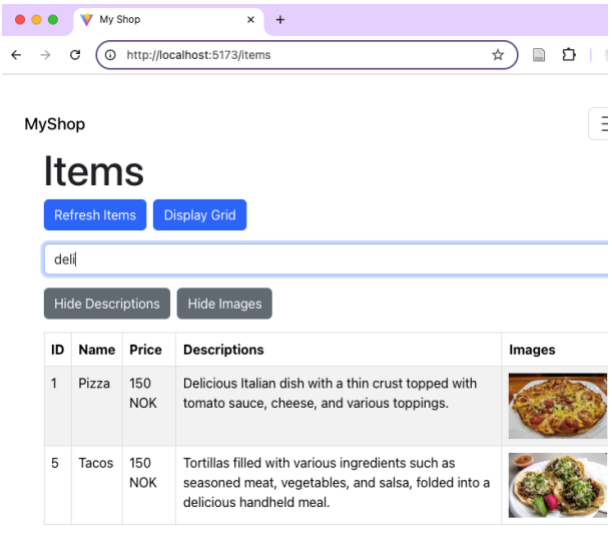
Line36-Line59:

```
36  return (
37    <div className="text-start">
38      <h1>Items</h1>
39      <Button onClick={() => refetch()} variant="primary" className="mb-3 me-2" disabled={isLoading}>
40        {isLoading ? 'Loading...' : 'Refresh Items'}
41      </Button>
42      <Button onClick={toggleTableOrGrid} variant="primary" className="mb-3 me-2">
43        {showTable ? `Display Grid` : 'Display Table'}
44      </Button>
45      <Form.Group className="mb-3">
46      <Form.Control
47        type="text"
48        placeholder="Search by name or description"
49        value={searchQuery}
50        onChange={e => setSearchQuery(e.target.value)}
51      />
52      </Form.Group>
53      {error && <p style={{ color: 'red' }}>Failed to fetch items.</p>}
54      {showTable
55      ? <ItemTable items={filteredItems} apiUrl={API_URL} />
56      : <ItemGrid items={filteredItems} apiUrl={API_URL} />}
57    </div>
58  );
59};
```

Line48-Line55, defines an input Form for inputting the **searchQuery**. The default text is rendered by Line51, and Line52 sets the value of the state **searchQuery** to the input text from the form once the input **Form** changes.
Line58-Line59: pass the **filteredItems** to the **ItemTable** or **ItemGrid**, not the complete array of items.

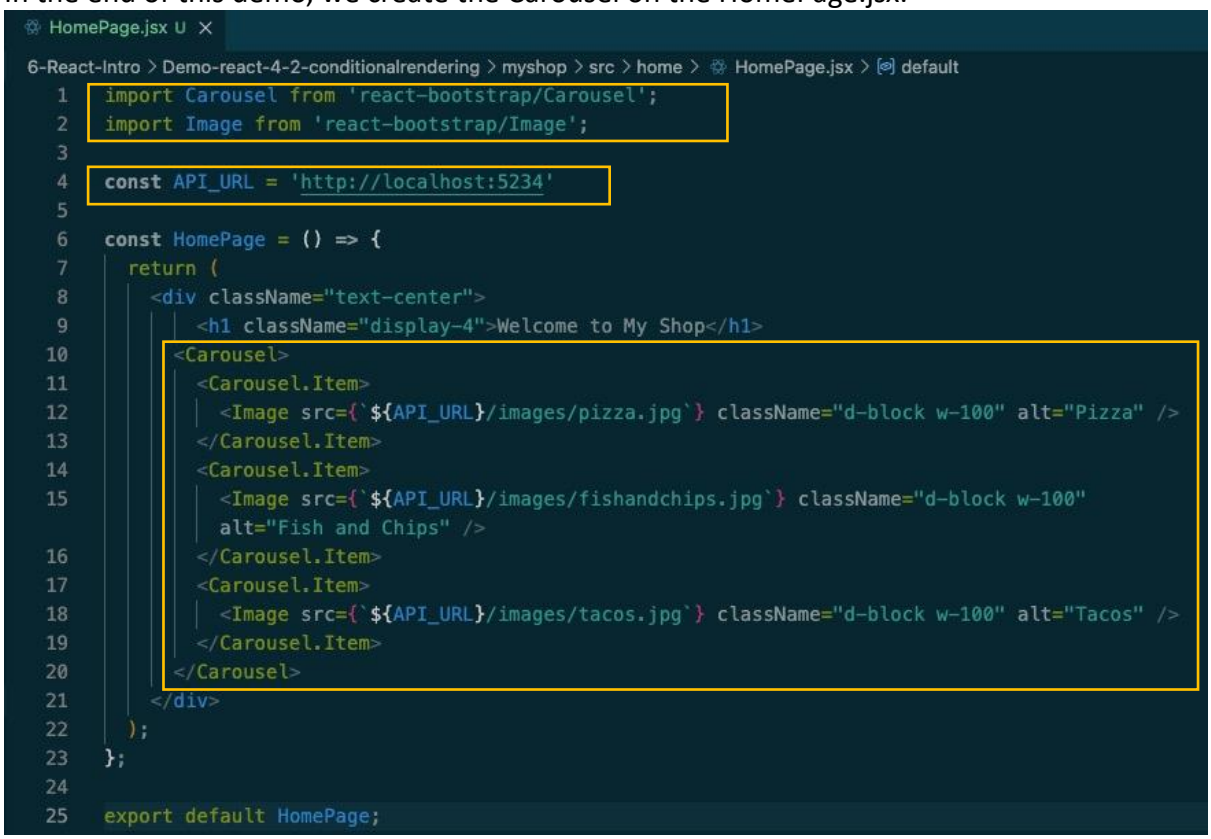
We see that the search bar is applied to both **ItemTable** and **ItemGrid**, because it is written on the **ItemListPage.jsx**, which contains the general logic for the components on the page.





Carousel in HomePage

In the end of this demo, we create the Carousel on the HomePage.jsx.



```jsx
import Carousel from 'react-bootstrap/Carousel';
import Image from 'react-bootstrap/Image';

const API_URL = 'http://localhost:5234'

const HomePage = () => {
  return (
    <div className="text-center">
      <h1 className="display-4">Welcome to My Shop</h1>
      <Carousel>
        <Carousel.Item>
          <Image src={`${API_URL}/images/pizza.jpg`} className="d-block w-100" alt="Pizza" />
        </Carousel.Item>
        <Carousel.Item>
          <Image src={`${API_URL}/images/fishandchips.jpg`} className="d-block w-100"
          alt="Fish and Chips" />
        </Carousel.Item>
        <Carousel.Item>
          <Image src={`${API_URL}/images/tacos.jpg`} className="d-block w-100" alt="Tacos" />
        </Carousel.Item>
      </Carousel>
    </div>
  );
};

export default HomePage;
```

Very convenient to use the Carousel and Image components from react-bootstrap.
You may notice that we need the **API_URL** again, which is quite cumbersome. We fill fix that later.

Congratulations! You have finished the demo of conditional rendering! ☺